

EmotionBudgetApp 프로젝트 구조 및 핵심

코드 설명

Android 프로그래밍 기말 프로젝트

김동민 / 2271180

1. 프로젝트 개요

EmotionBudgetApp 은 수입과 지출을 기록하면서 소비 당시의 감정도 함께 남길 수 있는 가계부 앱이다. 일반적인 가계부 기능에 감정 항목을 추가하여, 사용자가 어떤 감정일 때 소비를 많이 하는지 확인할 수 있도록 제작하였다.

기록 화면에서는 금액, 수입·지출 유형, 날짜, 카테고리, 감정, 메모를 입력한다. 통계 화면에서는 기록을 일별, 주별, 월별로 정리하며, 감정 화면에서는 감정별 지출 금액과 횟수를 그래프로 보여준다.

2. 개발 환경과 수업 내용 적용

2.1 개발 환경 및 라이브러리

사용 항목	사용 내용
Kotlin 2.2.10	앱의 데이터 처리와 화면 동작 코드를 작성하였다.
Android Gradle Plugin 9.1.1	Android 앱 모듈과 SDK 버전을 설정하였다.
Jetpack Compose	Kotlin 코드로 화면을 구성하였다.
Material3	버튼, 입력창, 카드, 하단 네비게이션 등의 UI 요소를 사용하였다.
Activity Compose	MainActivity 에서 Compose 화면을 실행하였다.
Lifecycle ViewModel Compose	화면에서 사용하는 기록 목록과 상태를 ViewModel 로 관리하였다.
StateFlow	기록 목록이 변경될 때 화면에 변경 내용이 반영되도록 사용하였다.

2.2 수업 내용 적용

수업에서 다룬 내용	프로젝트 적용 내용
Kotlin 변수, 조건문, 컬렉션	입력값을 변수로 관리하고 if , when , filter , map , sumOf 로 기록을 처리하였다.
객체지향 프로그래밍	Expense 데이터 클래스와 TransactionType 열거형으로 기록 형식을 정의하였다.
화면 구성과 이벤트 처리	입력창, 선택 메뉴, 버튼을 배치하고 onClick 이벤트로 기록 추가·수정·삭제를 처리하였다.
액티비티와 생명주기	MainActivity 에서 앱 화면을 시작하고 ViewModel 을 생성하였다.
할 일 목록 앱 구성	LazyColumn 으로 수입·지출 기록 목록을 표시하였다.
Android 라이브러리 활용	Jetpack Compose 와 Material3 컴포넌트를 사용하여 화면을 구성하였다.

3. 프로젝트 구조

```

EmotionBudgetApp/
├── app/
│   ├── build.gradle.kts
│   └── src/main/
│       ├── AndroidManifest.xml
│       ├── res/
│       └── java/com/example/emotionbudgetapp/
│           ├── MainActivity.kt
│           ├── data/
│           │   └── Expense.kt
│           ├── viewModel/
│           │   └── ExpenseViewModel.kt
│           └── ui/
│               ├── EmotionBudgetAppRoot.kt
│               ├── ExpenseScreen.kt
│               ├── ExpenseItem.kt
│               ├── LedgerReportScreen.kt
│               └── EmotionAnalysisScreen.kt
└── gradle/
    └── libs.versions.toml
  
```

파일	역할
app/build.gradle.kts	SDK 버전, Compose 사용 여부, 라이브러리 의존성을 설정한다.
gradle/libs.versions.toml	프로젝트에서 사용하는 라이브러리와 플러그인 버전을 관리한다.
MainActivity.kt	앱이 실행될 때 처음 호출되는 Activity 이다.
data/Expense.kt	수입·지출 기록 한 건의 데이터 형식을 정의한다.
viewModel/ExpenseViewModel.kt	기록 목록과 추가, 수정, 삭제, 합계 계산을 담당한다.
ui/EmotionBudgetAppRoot.kt	하단 탭을 이용해 기록, 통계, 감정 화면을 전환한다.
ui/ExpenseScreen.kt	기록 입력과 검색 조건을 처리하고 기록 목록을 표시한다.

파일	역할
ui/ExpenseItem.kt	기록 목록의 개별 항목을 카드 형태로 표시한다.
ui/LedgerReportScreen.kt	일별, 달력, 주별, 월별 통계 화면을 표시한다.
ui/EmotionAnalysisScreen.kt	감정별 지출 금액과 횟수를 계산하고 그래프로 표시한다.

4. 주요 기능

기능	설명
수입·지출 기록	금액, 유형, 날짜, 카테고리, 감정, 메모를 입력한다.
기록 수정·삭제	기존 기록을 다시 입력창에 불러오거나 삭제할 수 있다.
기록 검색	검색어, 기간, 유형, 카테고리, 감정, 금액 범위로 기록을 찾는다.
통계 화면	기록을 일별, 달력, 주별, 월별, 요약 형태로 정리한다.
감정 분석	감정별 총지출, 소비 횟수, 평균 금액과 전월 대비 횟수 변화를 보여준다.
감정 소비 진단	스트레스 소비의 횟수와 비중을 계산하여 진단 문구를 표시한다.

5. 핵심 코드 설명

5.1 앱 시작과 Compose 화면 호출

파일: `app/src/main/java/com/example/emotionbudgetapp/MainActivity.kt`

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            val expenseViewModel: ExpenseViewModel = viewModel()
            EmotionBudgetAppRoot(viewModel = expenseViewModel)
        }
    }
}
```

MainActivity 는 앱의 시작점이다. **onCreate()**에서 **setContent** 를 호출하여 Compose 화면을 표시한다. **viewModel()**로 생성한 **ExpenseViewModel** 은 기록 목록을 관리하며, 루트 화면에 전달되어 각 화면에서 함께 사용된다.

5.2 수입·지출 데이터 클래스

파일: `app/src/main/java/com/example/emotionbudgetapp/data/Expense.kt`

```
enum class TransactionType(val label: String) {
    INCOME("수입"),
    EXPENSE("지출")
}

data class Expense(
    val id: Int,
    val amount: Int,
    val category: String,
    val emotion: String,
    val memo: String,
    val dateMillis: Long,
    val type: TransactionType = TransactionType.EXPENSE
)
```

TransactionType 은 기록이 수입인지 지출인지 구분한다. **Expense** 는 기록 한 건에 필요한 값을 모아 둔 데이터 클래스이다. 목록에서 특정 기록을 수정하거나 삭제할 때는 **id** 를 사용하고, 날짜별·감정별 통계를 계산할 때는 **dateMillis** 와 **emotion** 값을 사용한다.

5.3 ViewModel 의 기록 목록 관리

파일: `app/src/main/java/com/example/emotionbudgetapp/viewmodel/ExpenseViewModel.kt`

```
private val _expenses = MutableStateFlow<List<Expense>>(emptyList())
val expenses: StateFlow<List<Expense>> = _expenses
private var nextId = 1

fun addExpense(amount: Int, category: String, emotion: String,
    memo: String, dateMillis: Long, type: TransactionType) {
    val newExpense = Expense(
        id = nextId++,
        amount = amount,
        category = category,
        emotion = emotion,
        memo = memo,
        dateMillis = dateMillis,
        type = type
    )
    _expenses.value = _expenses.value + newExpense
}

fun updateExpense(id: Int, amount: Int, category: String, emotion: String,
    memo: String, dateMillis: Long, type: TransactionType) {
    _expenses.value = _expenses.value.map { expense ->
        if (expense.id == id) {
            expense.copy(amount = amount, category = category, emotion = emotion,
                memo = memo, dateMillis = dateMillis, type = type)
        } else {
            expense
        }
    }
}

fun deleteExpense(expense: Expense) {
    _expenses.value = _expenses.value.filter { it.id != expense.id }
}
```

기록 목록은 **MutableStateFlow<List<Expense>>**로 관리한다. 기록 추가는 기존 목록 뒤에 새 기록을 붙이고, 수정은 **map** 과 **copy** 를 이용해 같은 **id** 의 기록만 바꾼다. 삭제는 **filter** 를 이용해 선택한 **id** 를 제외한 기록만 남긴다.

이 부분에는 Kotlin 컬렉션 처리와 객체지향 프로그래밍에서 다른 데이터 클래스 사용 방식이 적용되어 있다.

5.4 입력값 상태와 버튼 이벤트 처리

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/ExpenseScreen.kt`

```
var transactionType by remember { mutableStateOf(TransactionType.EXPENSE) }
var amountText by remember { mutableStateOf("") }
var category by remember { mutableStateOf("식비") }
var emotion by remember { mutableStateOf("기쁨") }
var memo by remember { mutableStateOf("") }
var editingExpenseId by remember { mutableStateOf<Int?>(null) }

ExpenseForm(
    amountText = amountText,
    onAmountChange = { amountText = it },
    onSubmit = {
        val amount = amountText.toIntOrNull()
        if (amount != null && amount > 0) {
            if (editingExpenseId == null) {
                viewModel.addExpense(
                    amount, category, emotion, memo,
                    selectedDateMillis, transactionType
                )
            } else {
                viewModel.updateExpense(
                    editingExpenseId!!, amount, category, emotion, memo,
                    selectedDateMillis, transactionType
                )
            }
            resetForm()
        }
    }
)
```

remember 와 **mutableStateOf** 는 사용자가 입력한 값을 화면 상태로 보관한다. 입력창의 값이 바뀌면 **onAmountChange** 같은 이벤트가 실행되어 상태가 변경된다.

기록 추가 버튼을 누르면 **onSubmit** 이 호출된다. 금액이 올바르게 입력되었는지 확인한 뒤, **editingExpenseId** 가 없으면 새 기록을 추가하고 값이 있으면 기존 기록을 수정한다.

5.5 LazyColumn 을 이용한 기록 목록

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/ExpenseScreen.kt`

```
val expenses by viewModel.expenses.collectAsState()

LazyColumn {
    items(filteredExpenses, key = { it.id }) { expense ->
        ExpenseItem(
            expense = expense,
            onEdit = {
                amountText = expense.amount.toString()
                category = expense.category
                emotion = expense.emotion
                memo = expense.memo
                editingExpenseId = expense.id
            },
            onDelete = {
                pendingDeleteExpense = expense
            }
        )
    }
}
```

collectAsState() 는 ViewModel 의 기록 목록을 화면에서 관찰한다. 기록이 추가되거나 삭제되면 목록이 바뀌고 화면도 다시 표시된다.

LazyColumn 은 화면에 필요한 항목만 그리기 때문에 기록이 많아져도 목록을 스크롤하여 볼 수 있다. 각 기록은 **ExpenseItem** 에 전달되어 수입·지출 유형, 금액, 날짜, 카테고리, 감정, 메모를 카드로 표시한다.

5.6 검색 조건을 이용한 기록 필터링

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/ExpenseScreen.kt`

```
val filteredExpenses = expenses.filter { expense ->
    val matchesQuery = appliedSearchText.isBlank() ||
        expense.type.label.contains(appliedSearchText, ignoreCase = true) ||
        expense.category.contains(appliedSearchText, ignoreCase = true) ||
        expense.emotion.contains(appliedSearchText, ignoreCase = true) ||
        expense.memo.contains(appliedSearchText, ignoreCase = true)

    val matchesType =
        typeFilter == "전체" || expense.type.label == typeFilter
    val matchesCategory =
        categoryFilter == "전체" || expense.category == categoryFilter
    val matchesEmotion =
        emotionFilter == "전체" || expense.emotion == emotionFilter
    val matchesAmount =
        (minAmountFilter == null || expense.amount >= minAmountFilter) &&
        (maxAmountFilter == null || expense.amount <= maxAmountFilter)

    matchesQuery && matchesType && matchesCategory &&
        matchesEmotion && matchesAmount
}
```

검색 기능은 Kotlin 의 **filter** 와 조건식을 사용한다. 검색어가 비어 있거나 기록의 유형, 카테고리, 감정, 메모에 검색어가 포함되면 검색어 조건을 만족한다. 유형, 카테고리, 감정, 최소·최대 금액 조건도 각각 계산한 뒤 모든 조건이 참인 기록만 결과 목록에 남긴다.

5.7 하단 탭을 이용한 화면 전환

파일: `app/src/main/java/com/example/emotionbudgetapp/ui/EmotionBudgetAppRoot.kt`

```
private enum class AppDestination(val label: String) {
    Ledger("기록"),
    Report("통계"),
    Emotion("감정")
}

var currentDestination by remember {
    mutableStateOf(AppDestination.Ledger)
}

Scaffold(
    bottomBar = {
        NavigationBar {
            AppDestination.values().forEach { destination ->
                NavigationBarItem(
                    selected = currentDestination == destination,
                    onClick = { currentDestination = destination },
                    icon = { Text(destination.label) }
                )
            }
        }
    }
) { innerPadding ->
    when (currentDestination) {
        AppDestination.Ledger -> ExpenseScreen(viewModel)
        AppDestination.Report -> LedgerReportScreen(expenses, onBack = {})
        AppDestination.Emotion -> EmotionAnalysisScreen(expenses, onBack = {})
    }
}
```

하단 네비게이션 항목을 누르면 **onClick** 에서 **currentDestination** 값이 바뀐다. **when** 문은 현재 선택된 값에 따라 기록, 통계, 감정 화면 중 하나를 표시한다. 화면을 기능별로 나누어 사용자가 원하는 기능으로 바로 이동할 수 있도록 구성하였다.

5.8 월별 통계와 감정별 통계 계산

파일: **app/src/main/java/com/example/emotionbudgetapp/ui/LedgerReportScreen.kt**,
EmotionAnalysisScreen.kt

```
val monthRecords = expenses.filter {
    it.dateMillis >= monthStart && it.dateMillis < monthEnd
}

val monthIncomeTotal = monthRecords
    .filter { it.type == TransactionType.INCOME }
    .sumOf { it.amount }

val monthExpenseTotal = monthRecords
    .filter { it.type == TransactionType.EXPENSE }
    .sumOf { it.amount }

val current = monthExpenses.filter { it.emotion == emotion }
val previous = previousMonthExpenses.filter { it.emotion == emotion }
val currentTotal = current.sumOf { it.amount }

EmotionStat(
    emotion = emotion,
    totalAmount = currentTotal,
    count = current.size,
    averageAmount = if (current.isEmpty()) 0 else currentTotal / current.size,
    previousCount = previous.size,
    countChange = current.size - previous.size
)
```

월별 통계는 선택한 달의 시작 시각과 다음 달 시작 시각 사이에 있는 기록만 **filter** 로 가져온다. 이후 수입과 지출을 나누어 **sumOf** 로 합계를 계산한다.

감정별 통계는 같은 감정의 기록을 모아 총금액, 횟수, 평균 금액을 계산한다. 이전 달의 기록 수도 함께 계산하여 감정별 소비 횟수가 증가했는지 확인할 수 있다.